

Tilburg University
Data Science Centre of Excellence

Robust search policy design for drone swarms in adversarial hide & seek environment

Jan van Gestel
ANR: 808440
SNR: 2123293
Supervisor: Prof. dr. ir. Roy Lindelauf
Submission date: **January 6, 2026**

Contents

1	Introduction	2
1.1	Choosing DSCE	2
1.2	What is the DSCE	3
1.3	Problem statement	3
2	Theoretical framework	3
3	Environment and Definitions	3
4	Searching Policies	4
5	Method	4
5.0.1	Simulation Environment	4
5.0.2	Experimental Design	4
6	Results	4
6.1	Performance by Swarm Size	5
6.2	Robustness over Environment	6
6.3	Policy Transition Matrix	6
7	Conclusion	6
8	Discussion	6
9	References	8
	References	8
10	Appendix	8
10.1	Searching policies	8
10.1.1	Game-Theoretic & Risk-Aware	8
10.1.2	Communication enabled	9
10.1.3	Cost-Optimization Heuristics	9
10.1.4	Geometric, Pattern based	10
10.1.5	Stochastic	10
10.2	Results	10
10.2.1	Hiders 1	10
10.2.2	Hiders 2	11
10.2.3	Hiders 5	12
10.2.4	Distributions	13
10.2.5	High var	13
10.2.6	High var	14
10.2.7	Skewed	14
10.2.8	Across swarm sizes	17

1 Introduction

1.1 Choosing DSCE

During my high school years, I developed a strong interest in the military and even trained extensively with the goal of enlisting as a Marine Corps officer. However, as time passed, the idea lost some of its initial appeal, and for several reasons, I ultimately decided not to pursue military enlistment. Despite this, my interest in the military never truly faded. As my knowledge of the AI domain grew, I became increasingly curious about how different organizations utilize AI in security contexts, and how contributions to safety and security are made. When I learned that the Data Science Centre of Excellence had recently been established and was closely related to my study program, the choice for my internship became clear.

1.2 What is the DSCE

The Data Science Centre of Excellence (DSCE) is a center of expertise for all aspects of Data Science and AI in the military domain. It operates as a cooperation between the Dutch Faculty of Military Sciences and the Chief Information Officer of the Ministry of Defence. The center’s mission is to develop and share Data Science and AI knowledge to enable safer and more efficient decision-making processes that contribute to safety and security. The DSCE operates on three main pillars:

- **Education:** enhancing data literacy for military personnel.
- **Operations:** applying AI models to real-world defense challenges (such as helicopter route planning).
- **Academia:** conducting fundamental research in collaboration with universities and knowledge institutes.

The core team reflects this multidisciplinary approach, consisting of a Head of Centre, a coordinator, (associate) professors, and military personnel. Beyond the core team, the center includes several PhD candidates and master’s and bachelor’s students who contribute to ongoing research.

1.3 Problem statement

Designing and identifying a effective and robust searching strategy for a drone swarm in a high risk environment.

2 Theoretical framework

Discrete search games were first established in (Bram, 1963), presenting a scenario where a hider selects one of n boxes for a target, and a searcher opens them sequentially to minimize the expected number of inspections (Lidbetter, 2025). This framework was expanded by (Lidbetter, 2020), who introduced the concept of a “success probability.” This addition models a dangerous environment where the searcher is not guaranteed to survive the expansion of a box, finding a theoretical solution for a single searching agent.

While drone swarms are a rapidly emerging technology that offers opportunities for search and rescue operations (Hoang et al., 2023; Arnold et al., 2020; Alfeo et al., 2018), existing research has not integrated this danger component. In high-stakes scenarios—such as navigating hazardous terrain or operating in adversarial environments with a risk of interception, accounting for agent loss is critical. Therefore, basing the approach on the single-searcher literature by (Lidbetter, 2020) has the highest potential of succeeding in finding a heuristic or even exact solution.

3 Environment and Definitions

Hiders and Strategies The game features n static hiders placed in a subset of candidate locations S . Candidate cells are selected uniformly at random, and hiding probabilities q_i are assigned based on three strategies: (1) **Greedy:** Hiders select the subset of cells with the highest q_a values (derived from (Lidbetter, 2020) Eq. 3); (2) **Weighted:** Hiders select cells probabilistically based on the distribution q_a (proven optimal for single-searchers); (3) **Random:** Probabilities q_i are drawn from a Dirichlet distribution such that $\sum q_i = 1$.

Success Probability (Risk) Each location $i \in S$ has a *success probability* p_i , representing the chance a searcher survives inspecting that cell. three risk environments were modeled:

- **High Variance:** $p_i \in \{0.10, 0.30, 0.60, 0.80, 0.95\}$;
- **Low Variance:** $p_i \in \{0.60, 0.62, 0.64, 0.66, 0.68\}$;
- **Skewed:** $p_i \in \{0.60, 0.62, 0.64, 0.66, 0.10\}$;

Swarm Dynamics The swarm enters the grid at (0,0) and operates under the following constraints: (1) Movement is restricted to cardinal directions (no diagonals); (2) Drones know candidate locations and success probabilities (p_i) but *not* the hiding probabilities (q_i) but tactics can of course re-calculate this; (3) Discovery is deterministic: if a drone expands a hider’s cell and survives, the hider is found ($P(\text{Found}|\text{Hider present} \wedge \text{Survival}) = 1$). Conversely, if the drone is taken down, the hider remains undiscovered.

4 Searching Policies

12 strategies across five categories were evaluated (see Appendix 10.1 for details): **Risk-aware:** Proportional risk split, Lidbetter, Weighted qa sampling, Ranked qa subset; **Communication:** Dynamic task queue; **Heuristic:** Shortest path, Risk weighted, Success weighted; **Geometric:** Horizontal scan, Spiral, Partitioned horizontal; **Baseline:** Random walk.

5 Method

5.0.1 Simulation Environment

A custom discrete-event simulator was developed using Python to evaluate the performance of drone swarm search policies. The simulator utilizes an object-oriented architecture to ensure modularity and code readability, allowing for distinct variations in search policies, hiding strategies, and environmental parameters. The simulation models an $N \times N$ grid world where movement is restricted to cardinal directions (North, East, South, West). To account for the stochastic nature of detection and the initialization of hider locations, a Monte Carlo simulation approach was adopted.

5.0.2 Experimental Design

The experiments were designed to test the performance of search policies under a wide variety of hiding conditions, employing the independent variables detailed in Table 1. A total of 100,000 simulation trials were conducted to yield the dependent variables. This sample size ensures statistical significance, resulting in a 95% confidence interval margin of error of $\epsilon \approx 0.005$ for the primary success metric, $P(\text{all found})$.

Table 1: Simulation parameters (Independent Variables).

Variable	Value(s)	Variable	Value(s)
Grid size	20×20	Hiders	$\{1, \dots, 5\}$
Swarm sizes	$\{1, 5, 10\}$	Strategies	$\{\text{Weighted, Random, Greedy}\}$
Hiding spots	5	Risk Profiles	$\{\text{Low, High, Skewed}\}$

Metrics and Evaluation five dependent variables were recorded: $P(\text{all found})$ (main performance metric), **Steps** (time to termination), **Mean distance**, **Fraction found**, and **Drones lost**.

A policy is considered a "win" if it achieves the highest $P(\text{all found})$ in a scenario. To assess robustness across the environments (E) and policies (Π), two specific criteria were utilized:

1. **Performance Spread:** Lower spread indicates higher stability.

$$\Delta P = \max_{\pi} P(\text{All Found}) - \min_{\pi} P(\text{All Found}) \quad (1)$$

2. **Maximin Criterion:** Selects the policy maximizing the worst-case outcome:

$$\pi^* = \operatorname{argmax}_{\pi \in \Pi} (\min_{e \in E} P(\text{All Found} | \pi, e)). \quad (2)$$

6 Results

Figure 1 illustrates the win counts across all simulated environments. A distinct trend correlates swarm size with policy dominance: single-drone settings favored the *Lidbetter* policy (16 wins), five-drone swarms favored *Proportional Risk Split* (34 wins), and ten-drone swarms were dominated by the *Dynamic Task Queue* (42 wins).

To identify the most robust strategies, the Maximin criterion (π^*_{maximin}) and Performance Spread metric (Eq. 1) were applied across three hider densities (1, 2, 5 hiders) and three risk profiles (Low Var, High Var, Skewed). Detailed numerical performance tables for all permutations are available in the Appendix 10.2 .

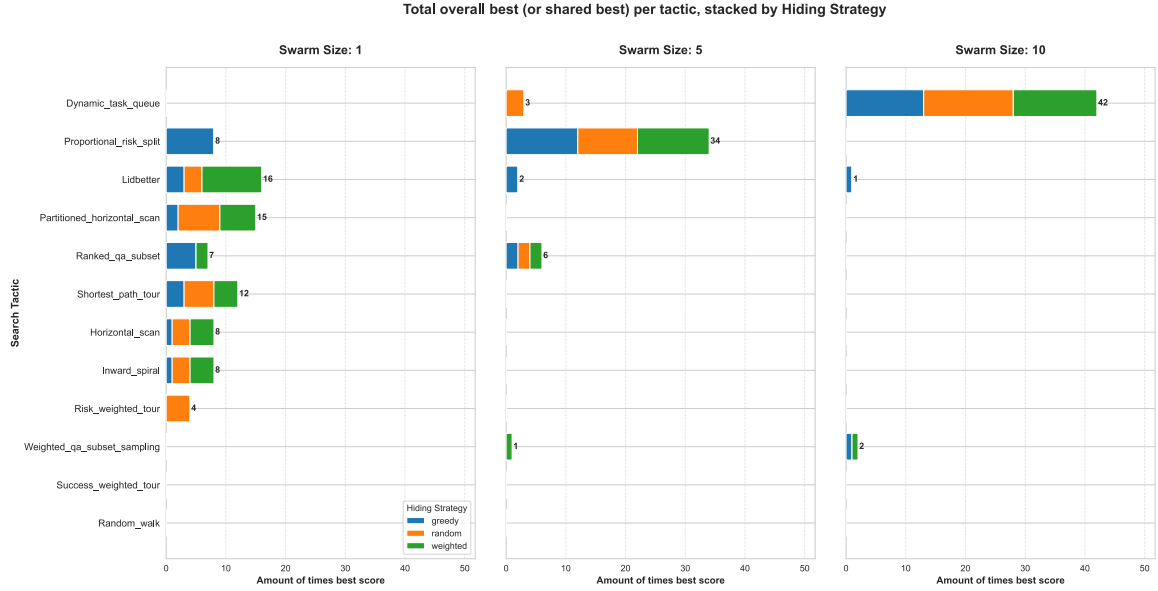


Figure 1: Stacked bar plot of policy win counts across all environmental conditions.

6.1 Performance by Swarm Size

Swarm Size 1 Across all hider densities, the *Lidbetter* policy consistently emerged as π_{maximin}^* . For 1 hider and 2 hidrs (Table 2), it clearly achieved the highest minimum $P(\text{All found})$ (success metric) (0.079, 0.025). This empirically validates the theoretical framework. However, regarding the saturated hider ratio, multiple searching policies yielded indistinguishable performance with a shared highest minimum. This suggests either an implementation anomaly or that performance discrepancies lie beyond the three-decimal precision threshold used in this analysis. While *Lidbetter* provided safety, exploitative strategies like *Ranked q_a Subset* achieved the absolute maximum (π_{max}) in specific low-risk conditions by prioritizing high-risk cells, effectively countering the *Greedy* hiding strategy. (Appendix, Fig. 2,3).

Swarm Size 5 As swarm size increased to 5, the optimal policy shifted away from pure game-theoretic approaches toward risk-aware heuristics. **1 Hider:** *Weighted q_a Subset Sampling* proved most robust (0.322). (Appendix, Fig. 2) **2 & 5 Hidrs:** *Proportional Risk Split* became dominant, achieving π_{maximin}^* in the 2 hider situation (0.161) and in the 5 hider situation (0.119). (Appendix, Fig. 3, Fig. 4)

This shift might indicate that with multiple agents, coordinating risk distribution becomes more critical than individual path optimization.

Swarm Size 10 **1 Hider:** Here *Weighted q_a Subset Sampling* proved robust again (0.542) indicating that this searching policy is the consistently the best to use in a 1 hider scenario, regardless of the swarm size. **2 & 5 Hidrs:** For larger swarms and situations with more than 1 hider, the *Dynamic Task Queue* was constantly superior. Dynamic Task Queue served as both π_{maximin}^* and π_{max} across all of these hider densities and risk profiles.

The ability to share information and dynamically reallocate targets seems to outweigh the benefits of static probabilistic planning in high-density swarms. (Appendix, Fig. 2, Fig. 3, Fig. 4)

Table 2: Summary of Optimal Robust Policies (π_{maximin}^*) by Swarm Size and Hider Count.

Hiders	Swarm Size 1	Swarm Size 5	Swarm Size 10
1	Lidbetter	Weighted q_a Subset	Dynamic Task Queue
2	Lidbetter	Prop. Risk Split	Dynamic Task Queue
5	Lidbetter / Geom.	Prop. Risk Split	Dynamic Task Queue

6.2 Robustness over Environment

The success probability distributions (Low Var vs. High Var vs. Skewed) clearly affected absolute success rates but rarely altered the *ranking* of the best policies. (Appendix, Fig. 5, Fig. 6, Fig. 7).

6.3 Policy Transition Matrix

The results suggest a deterministic decision logic for search and rescue operations. a transition matrix was constructed (Appendix Fig. 10, Fig. 11, Fig. 12, Fig. 13) to guide policy selection. 1. **Single Agent:** Deploy *Lidbetter*. 2. **Small Swarm ($N < 7$):** Transition to *Proportional Risk Split* or *Weighted q_a* . 3. **Large Swarm ($N \geq 7$):** Switch to *Dynamic Task Queue* to maximize efficiency.

These decisions remain mostly valid regardless of the specific environmental risk profile.

7 Conclusion

The performance and robustness of various swarm search policies were evaluated against adversarial hiding strategies in a grid world environment. Through extensive Monte Carlo simulations across varying swarm sizes, hider counts, hiding strategies, and success probability distributions, distinct transitions in optimal policy selection driven by swarm size and hider density were identified.

In single-hider scenarios, where the hider ratio is $1/5$, the optimal strategy for a single searcher is the game-theoretic optimum, specifically the *Lidbetter* searching policy. As the swarm size grows, the *Weighted q_a subset sampling* policy consistently emerges as the most robust choice. This indicates that the game-theoretic optimum is scalable; *Weighted q_a subset sampling* effectively applies the *Lidbetter* strategy individually across the drones in the swarm. While *Weighted q_a subset sampling* should also be the optimum for a single drone (as this effectively is *Lidbetter*). The implementation specifics regarding the avoidance of non-target cells in the actual *Lidbetter* and not applying this to the *Weighted q_a subset sampling* makes the latter more vulnerable to unnecessary risk and thus less effective.

In scenarios involving two hidens (hider ratio $2/5$), the optimal strategies shift away from *Weighted q_a subset sampling*. While the *Lidbetter* policy remains the most robust for a single searcher, it ceases to be optimal for swarms. For a medium-sized swarm of 5 agents, the *Proportional Risk Split* proves to be the safest policy, whereas for larger swarms of 10 agents, the *Dynamic Task Queue* dominates. This latter policy achieved both the highest worst-case ($\pi_{\max}^*$) and best-case ($\pi_{\min}$) performance across all distributions.

Similar trends were observed in the saturated five-hider scenario (ratio $5/5$), where hiding strategies became redundant. Although no clear single-searcher policy dominated, likely due to implementation artifacts rather than theoretical limitations, the swarm performance transitioned consistently: *Proportional Risk Split* was optimal for 5 drones, and *Dynamic Task Queue* was optimal for 10 drones. Notably, the specific success probability distribution of the environment did not seem to heavily alter the identification of the safest searching policy.

Synthesizing these results reveals a "turnaround" effect as swarm size grows, indicated by a predictable evolution in the optimal searching policy. For individual searchers ($N = 1$), the *Lidbetter* strategy represents the theoretical optimum. For small to mid-sized swarms ($N \approx 2 - 6$), the *Proportional Risk Split* becomes the most robust approach by effectively balancing risk-based allocation with static routing. Finally, for large swarms ($N \geq 7$), the *Dynamic Task Queue* becomes the dominant policy, as the benefits of active communication and dynamic reallocation outweigh the efficiency of static pre-planning. Ultimately, there is no single "one-size-fits-all" policy. Optimal search performance requires selecting a strategy based primarily on swarm size and secondarily on the expected number of targets, while the specific environmental probability distribution appears to be a less critical factor.

8 Discussion

The results of the analysis highlight distinct transitions in optimal policy selection based on swarm size. However, the analysis also revealed specific limitations in the current algorithm designs and identified promising avenues for future research to enhance robustness and performance.

Fusion of Dynamic Allocation and Risk Awareness A primary recommendation for future work is the fusion of the *Dynamic Task Queue* with game-theoretic, risk-aware probabilistic sampling. In the current implementation, the *Dynamic Task Queue* assigns targets largely random or based on proximity (Manhattan distance) without prioritizing safer or higher-value targets first. While this proved effective for large swarms where volume compensates for risk, it lacks efficiency for smaller swarms. By integrating the risk-based prioritization of the *Proportional Risk Split* into the dynamic assignment logic, the policy could potentially become the most robust solution across all swarm sizes, rather than just large ones.

(Multi-Agent) Reinforcement Learning While the report focused on heuristic and game-theoretic approaches, Multi-Agent Reinforcement Learning (MARL) presents a non-trivial alternative that should be investigated. RL agents have the potential to discover emergent coordination patterns that balance risk and coverage more effectively than static rule-based systems, potentially adapting dynamically to the specific probability distributions of the environment.

Non-Target Avoidance A performance issue was found between the *Lidbetter* strategy and *Weighted qa subset sampling* for single-agent scenarios, despite the fact that they are theoretically similar. The *Weighted qa subset sampling* was frequently outperformed because it prioritized the shortest route to the target subset, whereas the *Lidbetter* implementation explicitly included logic for "non-target avoidance" (avoiding potential hiding locations when this is not the target to avoid unnecessary risk). The non-target avoidance definitely has benefits and should therefore be implemented in all strategies that allow for this.

Risk Weighted Tour The *Risk Weighted Tour* performed poorly relative to other policies. A design flaw was identified in the cost function, which currently discounts distance based solely on the success probability of the *targeted* cell (P_{p2}), rather than accounting for the total survival probability of the entire route leading to that cell. To improve this strategy, the cost function should be updated to consider the total risk of the trajectory. Additionally, introducing a constant or dynamic factor to the discount mechanism could enhance the algorithm's ability to weigh safety against distance more effectively.

9 References

References

- Alfeo, A. L., Cimino, M. G., De Francesco, N., Lega, M., & Vaglini, G. (2018). Design and simulation of the emergent behavior of small drones swarming for distributed target localization. *Journal of Computational Science*, 29, 19-33. Retrieved from <https://www.sciencedirect.com/science/article/pii/S1877750318302898> doi: <https://doi.org/10.1016/j.jocs.2018.09.014>
- Arnold, R., Mezzacappa, E., Jablonski, J., & Abruzzo, B. (2020). Multi-role uav swarm behaviors for wide area search using emergent intelligence. In *2020 fourth world conference on smart trends in systems, security and sustainability (worlds4)* (p. 255-261). doi: 10.1109/WorldS450073.2020.9210392
- Bram, J. (1963). *A 2-player n-region search game* (Tech. Rep.).
- Hoang, M.-T. O., Grøntved, K. A. R., van Berkel, N., Skov, M. B., Christensen, A. L., & Merritt, T. (2023). Drone swarms to support search and rescue operations: Opportunities and challenges. In B. J. Dunstan, J. T. K. V. Koh, D. Turnbull Tillman, & S. A. Brown (Eds.), *Cultural robotics: Social robots and their emergent cultural ecologies* (pp. 163-176). Cham: Springer International Publishing. Retrieved from https://doi.org/10.1007/978-3-031-28138-9_11 doi: 10.1007/978-3-031-28138-9_11
- Lidbetter, T. (2020, 2). Search and rescue in the face of uncertain threats. *European Journal of Operational Research*, 285(3), 1153-1160. Retrieved from <https://doi.org/10.1016/j.ejor.2020.02.029> doi: 10.1016/j.ejor.2020.02.029
- Lidbetter, T. (2025). A review of minimum cost box searching games. *arXiv preprint arXiv:2502.10551*.

10 Appendix

10.1 Searching policies

10.1.1 Game-Theoretic & Risk-Aware

Divide over risk - Proportional Risk split This strategy allocates the drone swarm to candidate locations proportionally to the risk associated with each location, defined as ‘1 - probability of success’. The goal is to send the largest groups of drones to the highest-risk locations first.

First, all candidate locations are sorted by risk. Then, a fractional number of drones is calculated for each location based on its proportional risk (e.g., 7.44 drones for location A, 2.56 for B). The algorithm assigns the *floor* of this number to each location (7 to A, 2 to B). The remaining drones (the “remainder”) are then assigned one by one to the locations with the largest fractional parts (B would get the first remainder, as 0.56 > 0.44).

In the routing phase, all drones are assigned to visit all candidate locations. Each drone group’s route is a `visit_order` list that is rotated to start with that group’s primary assigned target. For example, if the ordered risk locations are [A, B, C]:

- Group A (highest risk): [A, B, C]
- Group B (mid risk): [B, C, A]
- Group C (low risk): [C, A, B]

Lidbetter In this searching strategy the cells are explored based on the optimal searching strategy given by (Lidbetter, 2020). A subset of possible hiding locations is chosen with probability q_a , after searching those, the remaining possible hiding locations are chosen uniformly at random. The swarm traverses all cells together. ($A \in S^{(k)}$) with probability

$$q_A = \lambda_k \prod_{i \in A} \frac{1 - p_i}{p_i}, \text{ where } \lambda_k = \left(\sum_{B \in S^{(k)}} \prod_{i \in B} \frac{1 - p_i}{p_i} \right)^{-1}. \quad (3)$$

Contrary to other strategies the Lidbetter strategy was implemented to specifically try to find a route that does not come across any of the possible hiding locations that are not the main target.

Traverse P q_a , Weighted q_a subset sampling This strategy independently samples a subset $A \in S^{(k)}$ for each drone according to the distribution q_a from Equation 3. This then is its first objective, where after it

then traverses all remaining candidate cells at uniform random order. Such that its complete route consists of all possible hiding locations. Whereas the Lidbetter policy in 10.1.1 has the swarm collectively choose a single subset according to q_a and then visit the remainder uniformly at random together, this variant applies the same distribution per drone.

Traverse ordered qa, Ranked qa subset This strategy assigns each drone to a different subset $A \in S^{(k)}$ based on the q_a values from Equation 3. All subsets are sorted in descending order by q_a , creating an ordered list where index 0 corresponds to the highest-value subset.

Drones are assigned sequentially: drone 1 receives the subset at index 0 (highest q_a), drone 2 receives index 1, and so on. If the number of drones exceeds the number of subsets, the assignment wraps around. For example, with 4 drones and 3 subsets, drone 4 would be assigned the subset at index 0 again.

After receiving its primary subset, each drone visits those cells first, then visits all remaining candidate locations in order of the list, like the primary target assignment wrapping around if the list gets exhausted.

10.1.2 Communication enabled

Shared list, Dynamic task queue Different from the other policies that are given an initial route and then fly it, this searching strategy dynamically assigns new locations. I.e. allowing the drones to communicate during the mission. Let S be the set of possible hiding locations. Initially, up to $\min(\text{swarm_size}, |S|)$ drones are assigned to hiding locations from S , leaving the remaining drones idle. The active and idle states are tracked, along with a 'to search' queue and a set of successfully searched locations. When a drone is assigned a location, that location is removed from the 'to search' queue.

If a drone dies either during route traversal or while searching the location, that location is appended back to the 'to search' queue. The closest idle agent (measured by Manhattan distance) is then assigned this target. The route given to this reassigned agent avoids successfully searched locations.

When a drone successfully completes its route and searches a location, that location is added to the successfully searched set, and the drone returns to the idle state where it can be reassigned to remaining locations in the queue.

Note that there is no priority given to cells with a higher $1 - p_i$ such that these are searched first. This could be a future improvement such that smaller sized swarms are more successful.

10.1.3 Cost-Optimization Heuristics

Together traverse best permutation, Shortest path tour This strategy calculates the optimal permutation of cells to visit, such that the total distance traveled is minimized. Given that the environment is a grid that does not allow diagonal movement, the manhattan distance formula is used to calculate the distance: $|x_1 - x_2| + |y_1 - y_2|$. The simulated implementation brute forces all options, and does not use any of the Traveling Salesman Problem optimizations. The swarm traverses all cells together.

Discounted Distance - Risk weighted tour This strategy selects the permutation of candidate cells that minimizes the total travel costs over the entire route, i.e., it takes the permutation whose total cost is minimal under the cost function below. Given consecutive cells $p_1 = (x_1, y_1)$ and $p_2 = (x_2, y_2)$, the travel cost is

$$\text{cost}(p_1, p_2) = (|x_1 - x_2| + |y_1 - y_2|) \times (1 - P_{p_2}) \quad (4)$$

where the distance term is the Manhattan metric and P_{p_2} is the success probability at the next cell; the total route cost is the sum of these costs, and the implementation brute-forces all permutations to find the minimum-sum one with the swarm traversing together.

Discounted Distance Reverse - Success weighted tour This policy is almost identical to 10.1.3 Yet now the discount factor is P_{p_2} , yielding the cost function:

$$\text{cost}(p_1, p_2) = (|x_1 - x_2| + |y_1 - y_2|) \times P_{p_2} \quad (5)$$

The simulated implementation brute forces all options, and does not use any of the Traveling Salesman Problem optimizations. The swarm traverses all cells together.

10.1.4 Geometric, Pattern based

Horizontal scan traversal - Horizontal scan In this searching strategy drones collectively traverses the complete grid starting from Top-left and ending on the bottom-left. This is done in the order: left to right, down, right to left, down, repeat. Thus making a 'snake-like' movement pattern.

Spiral scan traversal - Inward Spiral In this searching strategy drones collectively traverse the grid in a spiraling pattern. Moving together from top left to bottom left, to bottom right, top right, back to the top left and going inwards right before the first cell. Continuing this movement until all cells have been expanded.

Partitioned horizontal scan traversal - Partitioned horizontal scan This searching strategy is similar to the horizontal scan in 10.1.4, yet drones now independently traverse the grid from Top-left to bottom-left. They too start at the top left, but continue to divide themselves (as evenly as possible) over the vertical axis. Effectively scanning the board in partitions, making the same snake like pattern, but starting the movement at different parts of the board.

10.1.5 Stochastic

Random Walk In this search policy every drone can , at each time step t , independently choose to move in one of the North, East, South, West directions in uniform random order, restricted by the bounds of the grid.

10.2 Results

10.2.1 Hiders 1

Prob. Dist.	Metric	Swarm size 1	Swarm size 5	Swarm size 10
Low var	$\pi_{\max}^*$	Lidbetter: 0.315	Ranked Q_a subset & proportional risk split: 0.852	Dynamic Task Queue: 0.985
	$\pi_{\max}$	Proportional risk split: 0.562	Ranked Q_a subset: 0.864	Dynamic Task Queue: 0.988
High var	$\pi_{\max}^*$	Lidbetter: 0.079	Weighted Q_a subset sampling: 0.322	Weighted Q_a subset sampling: 0.542
	$\pi_{\max}$	Risk Weighted Tour: 0.241	Dynamic Task Queue: 0.632	Dynamic Task Queue: 0.822
Skewed	$\pi_{\max}^*$	Lidbetter: 0.086	Weighted Q_a subset sampling: 0.344	Weighted Q_a subset sampling: 0.567
	$\pi_{\max}$	Risk Weighted Tour: 0.209	Ranked Q_a subset: 0.648	Dynamic Task Queue: 0.845

Table 3: Tabular view of the tactic $\pi_{\max}^*$ and $\pi_{\max}$ and the corresponding $P(\text{all found})$ value for 1 hider, hider ratio: $\frac{1}{5}$

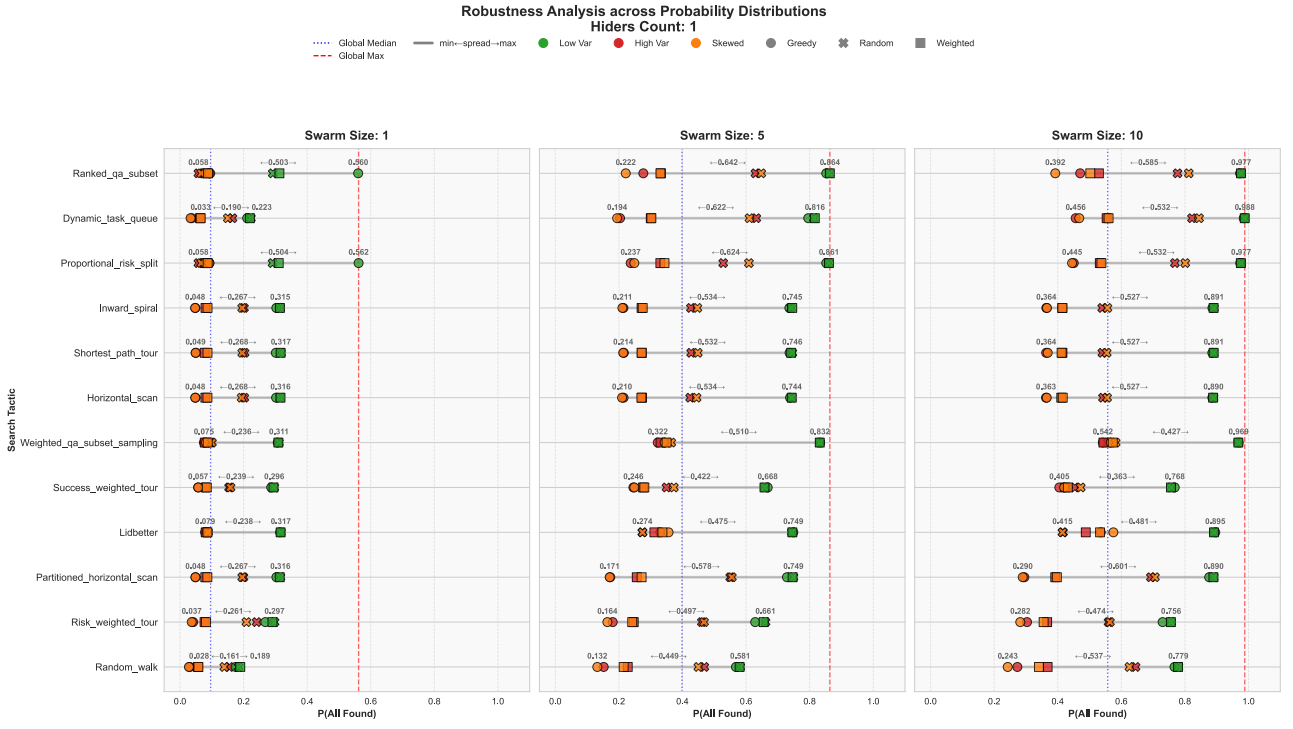


Figure 2: Robustness spread (Cleveland dot plot) for Swarm Sizes 1, 5, and 10 (1 Hider case). Narrower spreads indicate higher robustness.

10.2.2 Hiders 2

Prob. Dist.	Metric	Swarm size 1	Swarm size 5	Swarm size 10
Low var	$\pi_{\max}^*$	Lidbetter: 0.182	Proportional risk split: 0.760	Dynamic Task Queue: 0.976
	$\pi_{\max}$	Proportional risk split: 0.345	Proportional risk split: 0.773	Dynamic Task Queue: 0.979
High var	$\pi_{\max}^*$	Lidbetter: 0.025	Proportional risk split: 0.161	Dynamic Task Queue: 0.395
	$\pi_{\max}$	Risk Weighted Tour: 0.084	Dynamic Task Queue: 0.401	Dynamic Task Queue: 0.681
Skewed	$\pi_{\max}^*$	Lidbetter: 0.017	Proportional Risk split: 0.204	Dynamic Task Queue: 0.445
	$\pi_{\max}$	Risk Weighted Tour: 0.077	Proportional Risk split: 0.407	Dynamic Task Queue: 0.719

Table 4: Tabular view of the tactic $\pi_{\max}^*$ and $\pi_{\max}$ and the corresponding $P(\text{all found})$ value for 2 hiders, hider ratio : $\frac{2}{5}$

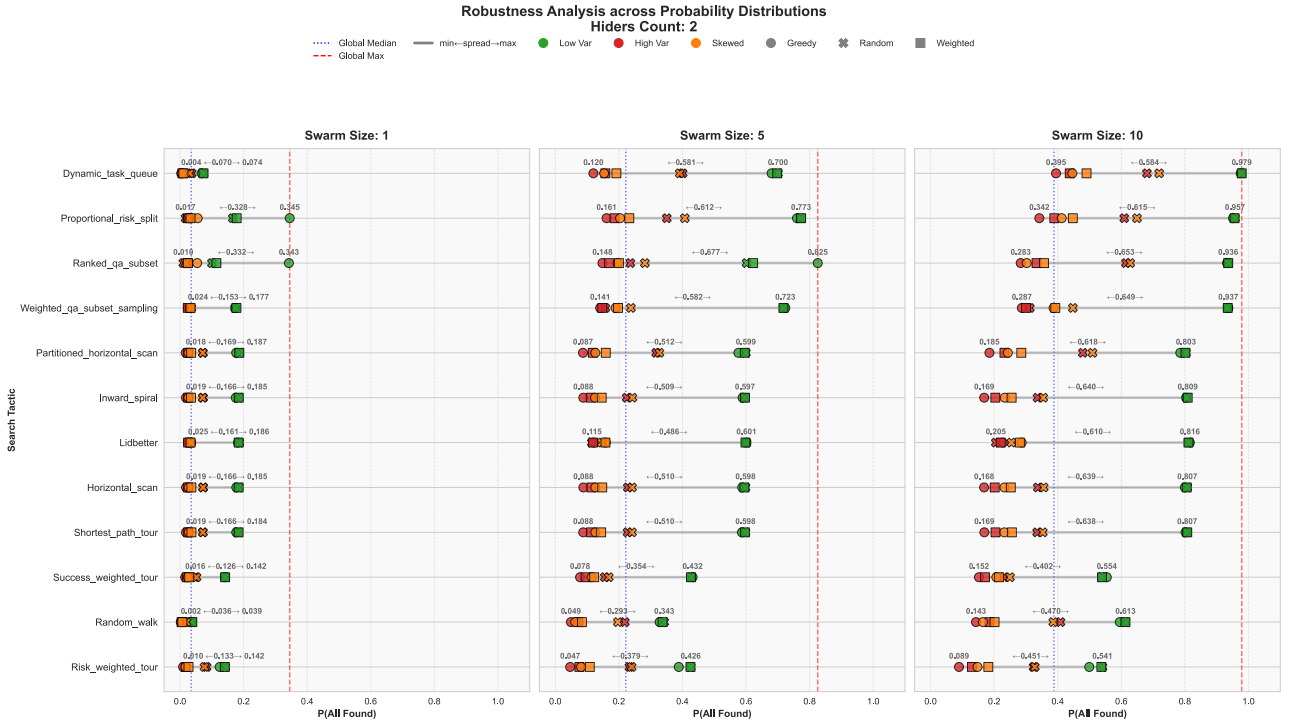


Figure 3: Cleveland dot plot of probability of all hidens found for swarm size 1, 5 and 10, given 2 hidens, 5 possible hiding locations, simulated on a 20 x 20 grid. The plot shows robustness of strategies across the different probability distributions and different hiding strategies, i.e. wider lines could be viewed as less robust.

10.2.3 Hiders 5

Prob. Dist.	Metric	Swarm size 1	Swarm size 5	Swarm size 10
Low var	$\pi_{\max}^*, \pi_{\max}$	Lidbetter, shortest path tour, (Partitioned) horizontal scan: 0.106	Proportional risk split: 0.625	Dynamic Task Queue: 0.961
High var	$\pi_{\max}^*, \pi_{\max}$	Lidbetter, shortest path tour, (Partitioned) horizontal scan, inward spiral: 0.013	Proportional risk split: 0.120	Dynamic Task Queue: 0.389
Skewed	$\pi_{\max}^*, \pi_{\max}$	Lidbetter, shortest path tour, (Partitioned) horizontal scan, inward spiral: 0.015	Proportional Risk split: 0.142	Dynamic Task Queue: 0.430

Table 5: Tabular view of the tactic $\pi_{\max}^*$ and $\pi_{\max}$ and the corresponding $P(\text{all found})$ value for 2 hidens, hider ratio : $\frac{2}{5}$

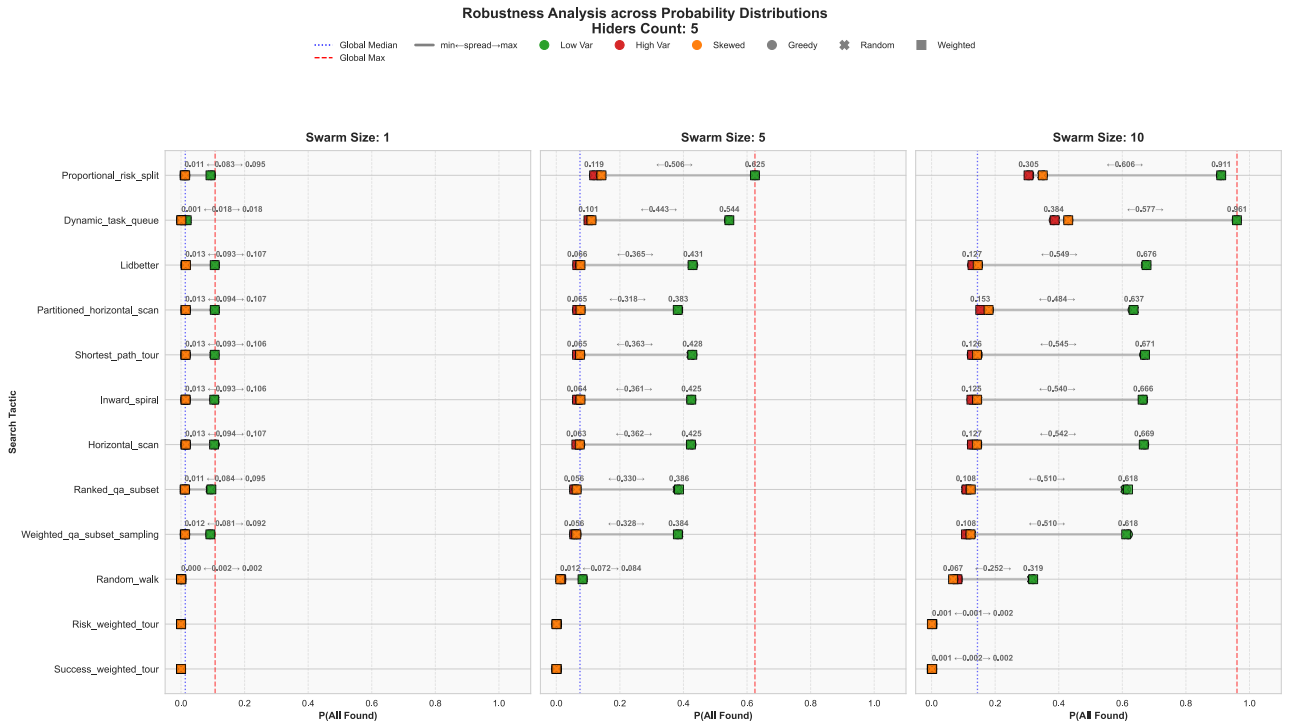


Figure 4: Cleveland dot plot of probability of all hiders found for swarm size 1, 5 and 10, given 5 hiders, 5 possible hiding locations, simulated on a 20 x 20 grid. The plot shows robustness of strategies across the different probability distributions and different hiding strategies, i.e. wider lines could be viewed as less robust.

10.2.4 Distributions

10.2.5 High var

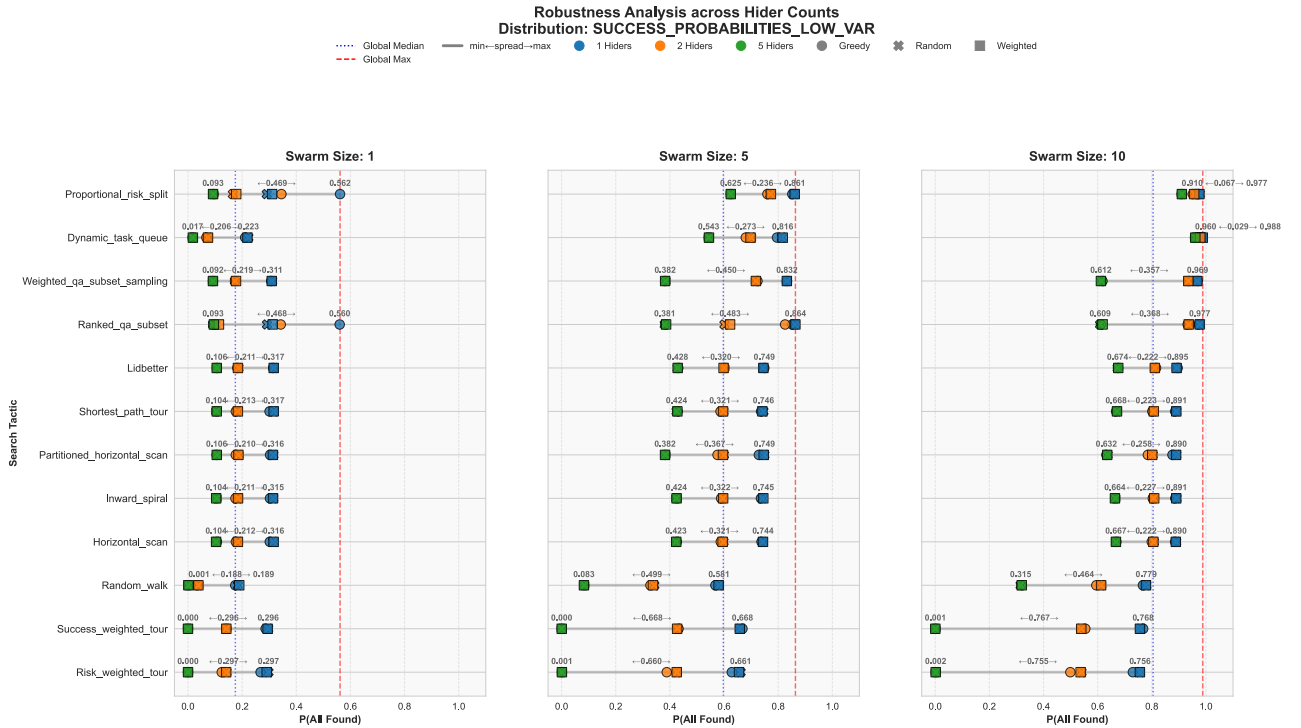


Figure 5: Cleveland dot plot of probability of all hiders found for swarm size 1, 5 and 10, Probability distribution LOW VAR, simulated on a 20 x 20 grid. The plot shows robustness of strategies across the different hider ratios and different hiding strategies, i.e. wider lines could be viewed as less robust.

10.2.6 High var

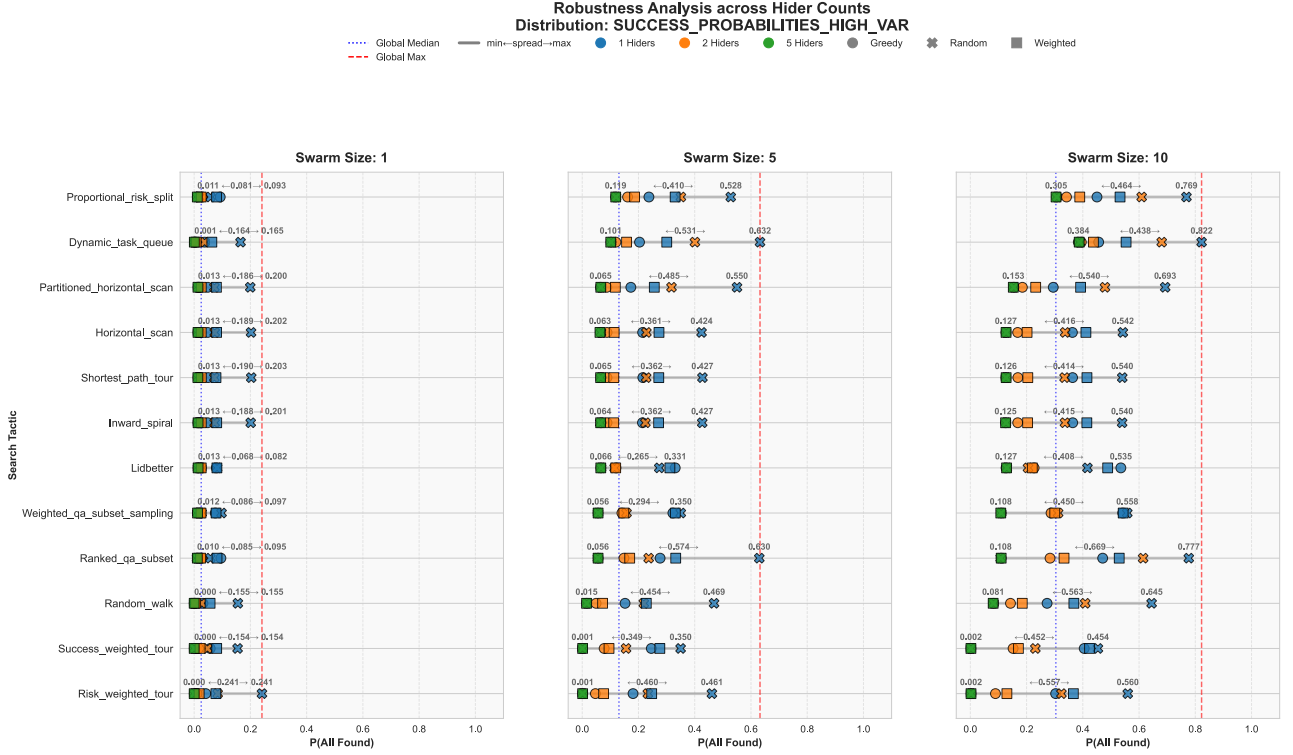


Figure 6: Cleveland dot plot of probability of all hiders found for swarm size 1, 5 and 10, Probability distribution HIGH VAR, simulated on a 20 x 20 grid. The plot shows robustness of strategies across the different hider ratios and different hiding strategies, i.e. wider lines could be viewed as less robust.

10.2.7 Skewed

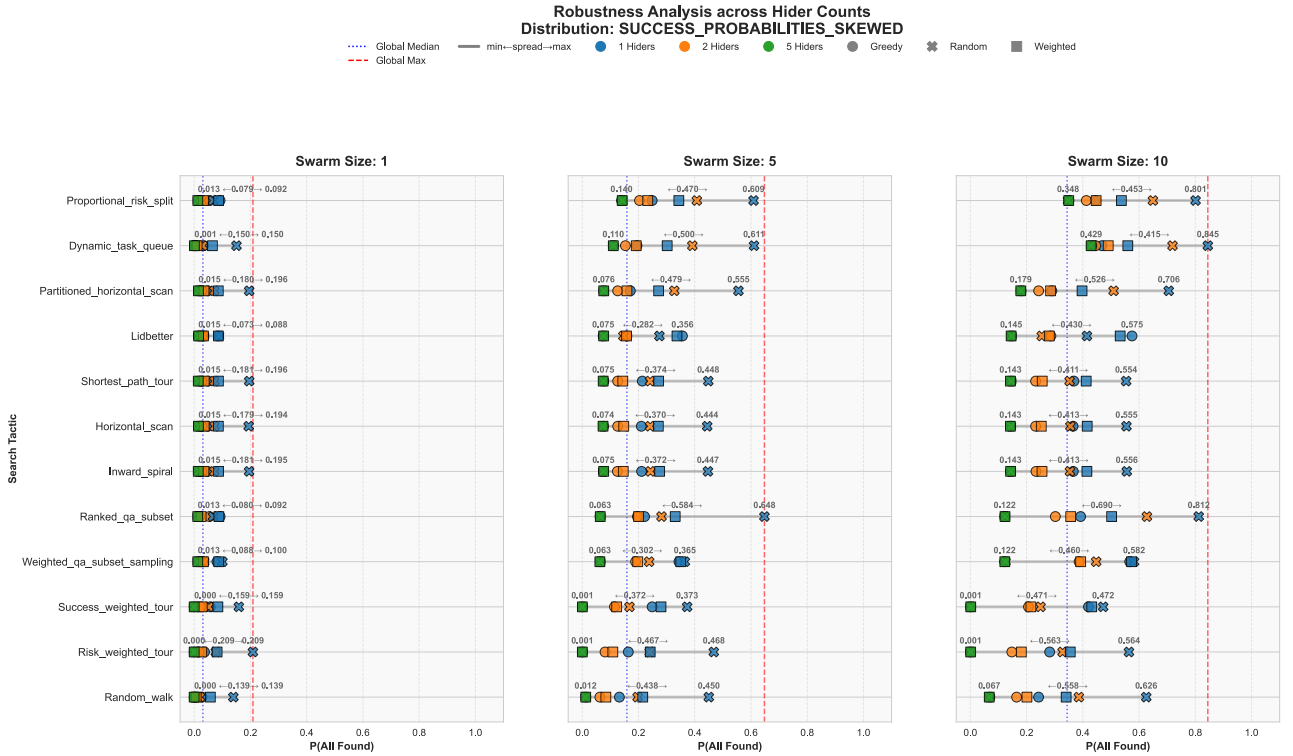
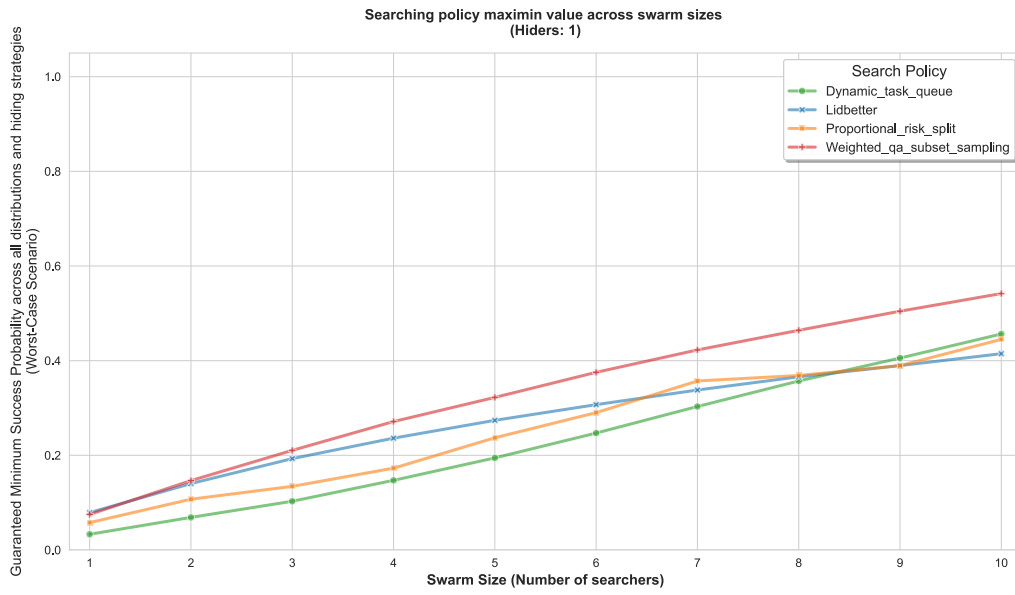


Figure 7: Cleveland dot plot of probability of all hiders found for swarm size 1, 5 and 10, Probability distribution SKEWED, simulated on a 20 x 20 grid. The plot shows robustness of strategies across the different hider ratios and different hiding strategies, i.e. wider lines could be viewed as less robust.

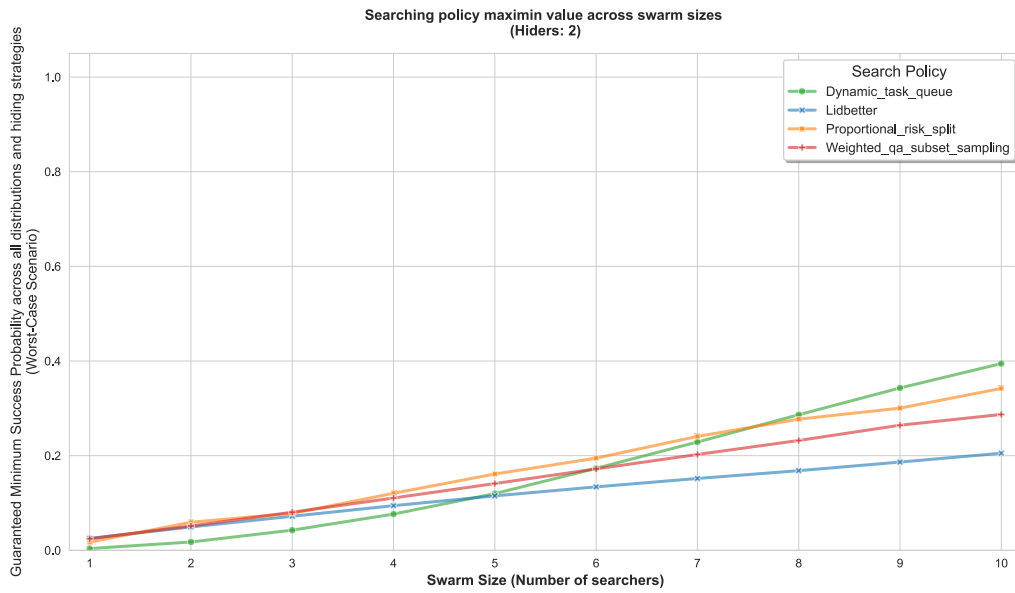
Prob. Dist.	Metric	Swarm size 1	Swarm size 5	Swarm size 10
Low var	$\pi_{\max}^*$	Lidbetter & (partitioned)horizontal scan: 0.106,	proportional risk split: 0.625	Dynamic Task Queue: 0.960
	$\pi_{\max}$	Proportional risk split: 0.562	Ranked Q_a subset: 0.864	Dynamic Task Queue: 0.988
High var	$\pi_{\max}^*$	Lidbetter,shortest path tour, (Partitioned) horizontal scan, inward spiral: 0.013	proportional risk split: 0.119	Dynamic Task Queue: 0.384
	$\pi_{\max}$	Risk Weighted Tour: 0.241	Dynamic Task Queue: 0.632	Dynamic Task Queue: 0.822
Skewed	$\pi_{\max}^*$	Lidbetter, shortest path tour,(Partitioned) horizontal scan, inward spiral: 0.015	proportional risk split: 0.140	Dynamic Task Queue: 0.429
	$\pi_{\max}$	Risk Weighted Tour: 0.209	Ranked Q_a subset: 0.648	Dynamic Task Queue: 0.845

Table 6: Tabular view of the tactic $\pi_{\max}^*$ and $\pi_{\max}$ and the corresponding $P(\text{all found})$ value for the different probability distributions. Showing all hider options.

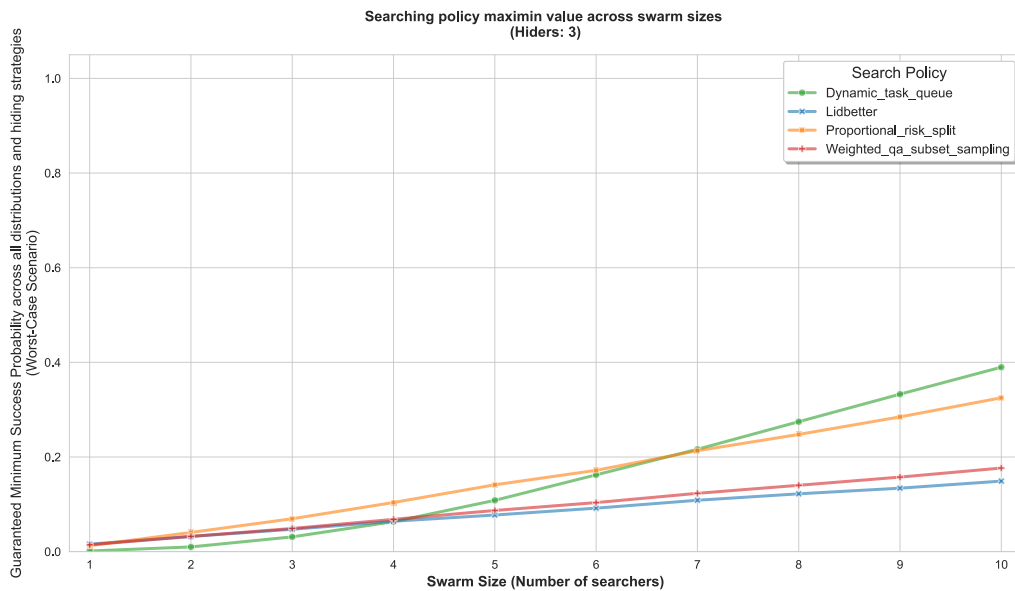
10.2.8 Across swarm sizes



(a) Hiders: 1

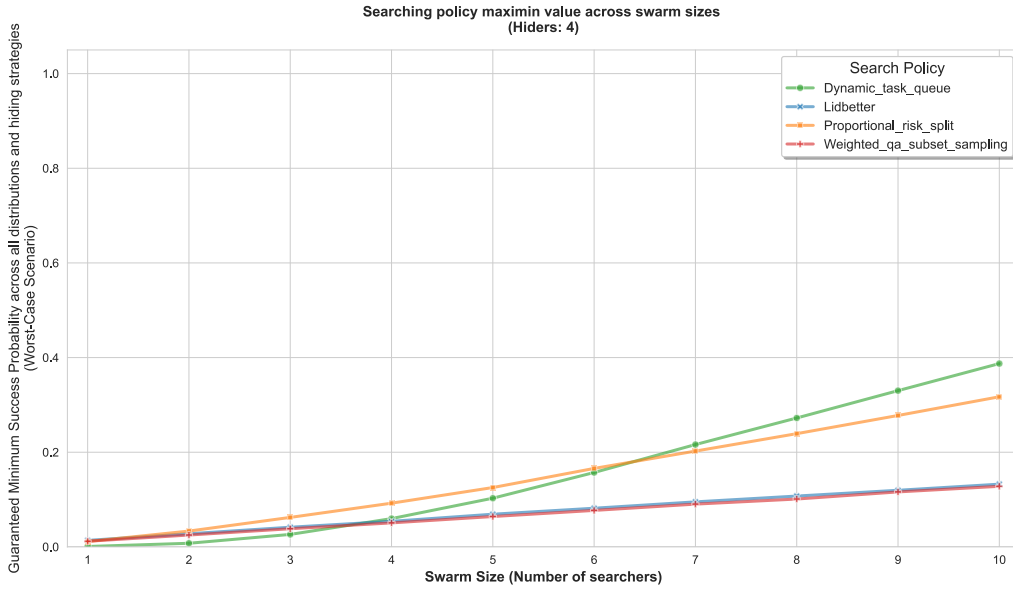


(b) Hiders: 2

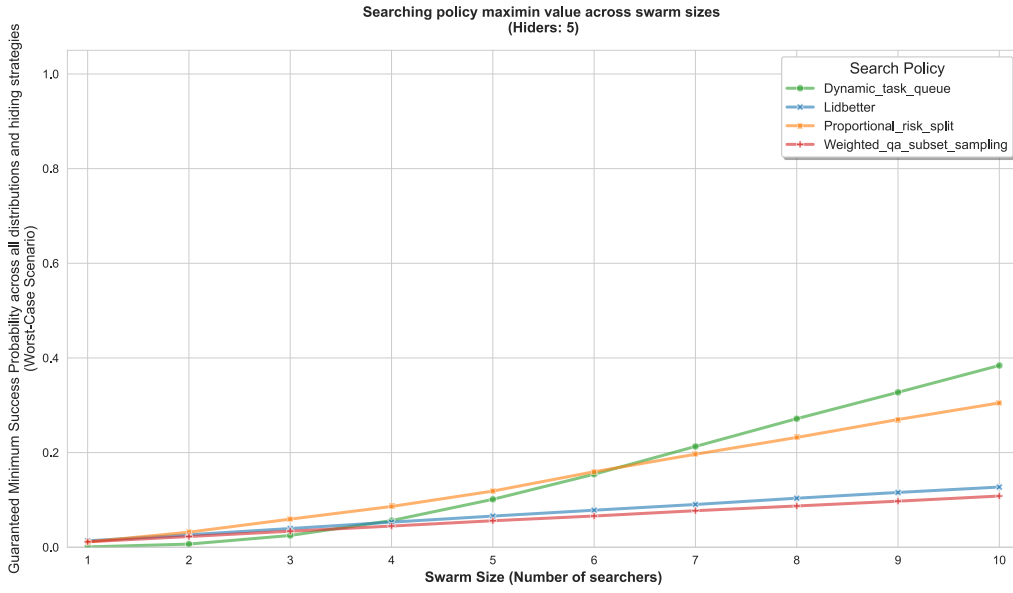


(c) Hiders: 3

Figure 8: Turnaround analysis (Part 1 of 2). Strategies for smaller hider counts show distinct performance gaps.



(d) Hiders: 4



(e) Hiders: 5

Figure 8: Turnaround analysis (Part 2 of 2). As the number of hiders increases to 4 and 5, the performance of the Dynamic Task Queue improves relative to Weighted QA.

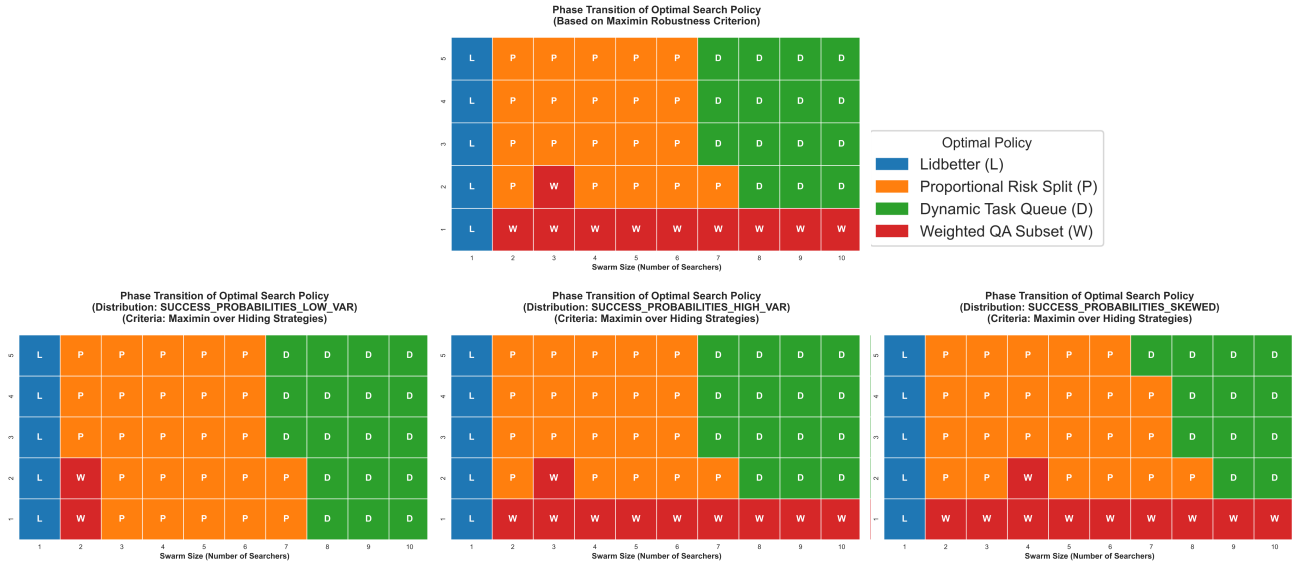


Figure 9: Transition matrix of the optimal search policy for a given swarm size (horizontal axis, 1 – 10), and a given number of hiders (vertical axis, 1 – 5), regardless of hiding strategy. Top plot is regardless of probability distribution, bottom three from left to right is specified for the: Low variation, High variation, skewed probability distributions.

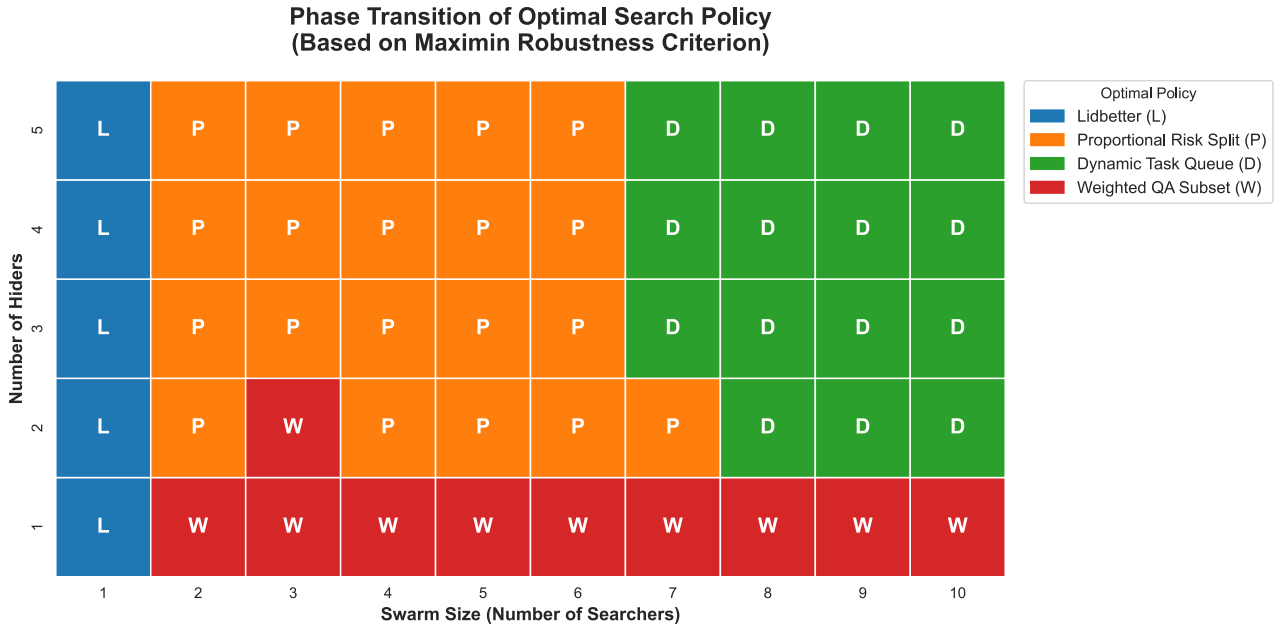


Figure 10: Transition matrix of the optimal search policy for a given swarm size (horizontal axis, 1 – 10), and a given number of hiders (vertical axis, 1 – 5), regardless of hiding strategy & probability distribution.

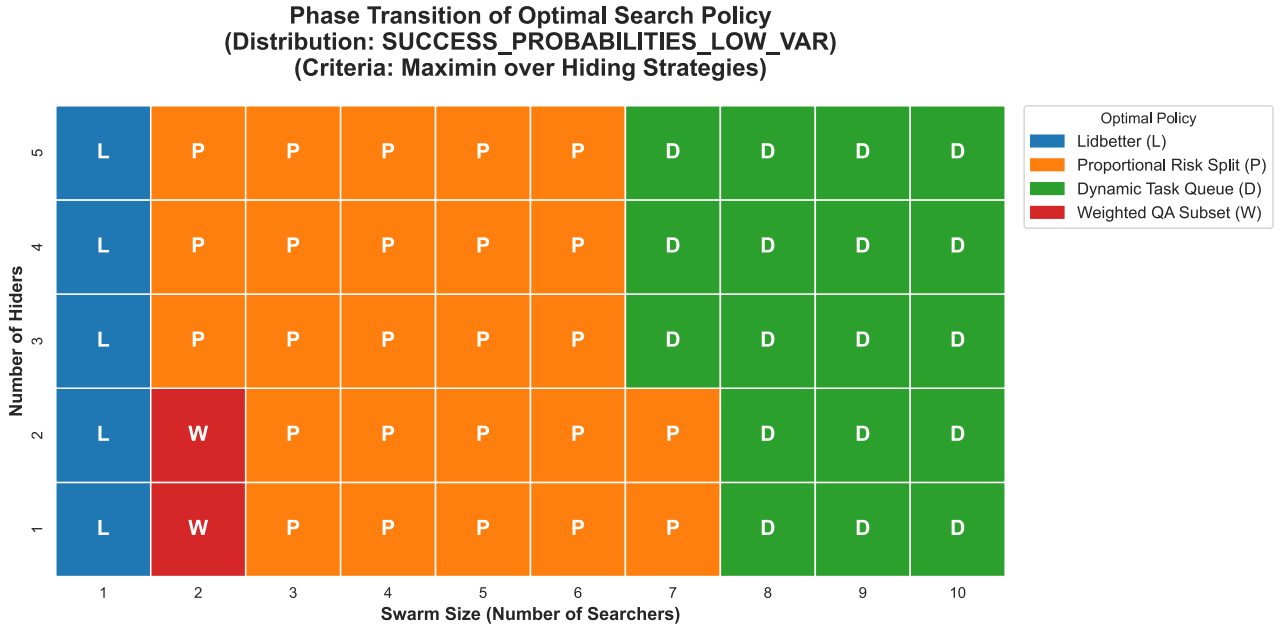


Figure 11: Transition matrix of the optimal search policy for a given swarm size (horizontal axis, 1 – 10), and a given number of hiders (vertical axis, 1 – 5), regardless of hiding strategy. specified for the: Low variation probability distribution.

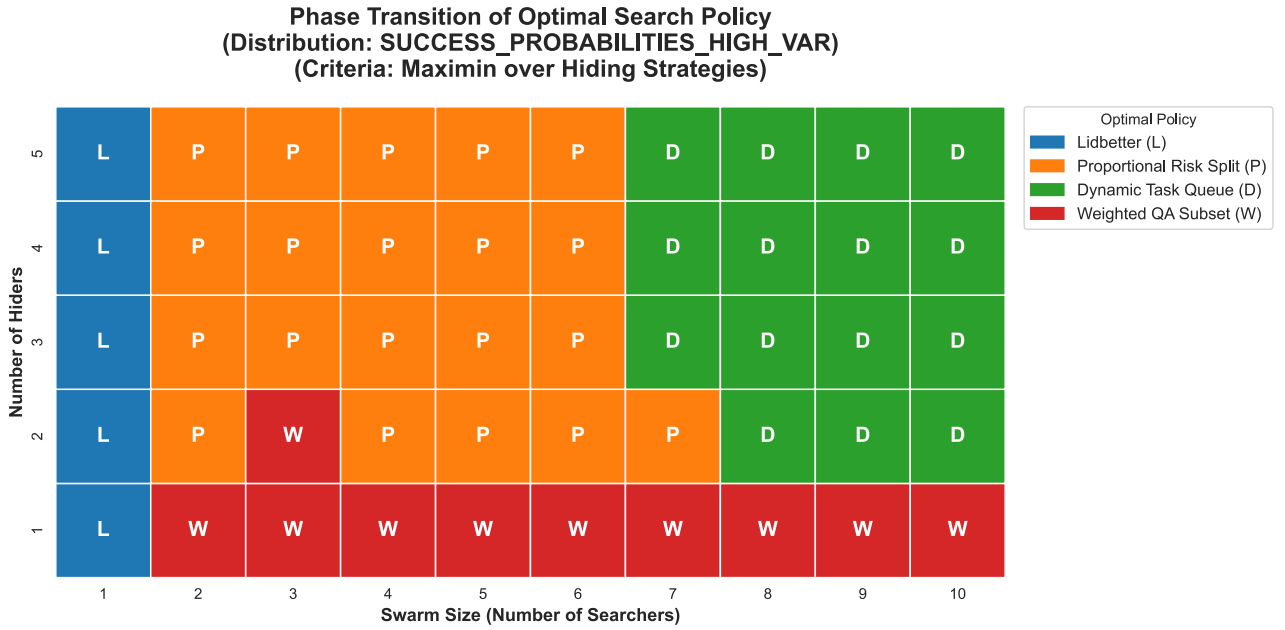


Figure 12: Transition matrix of the optimal search policy for a given swarm size (horizontal axis, 1 – 10), and a given number of hiders (vertical axis, 1 – 5), regardless of hiding strategy. specified for the: High variation probability distribution.

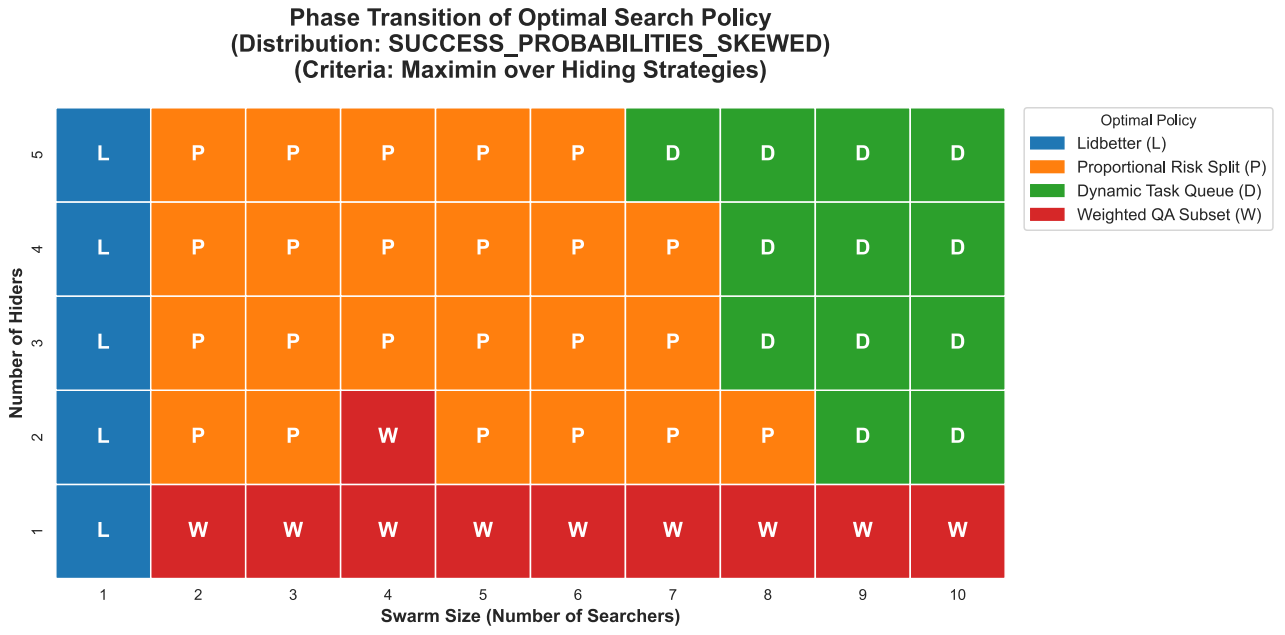


Figure 13: Transition matrix of the optimal search policy for a given swarm size (horizontal axis, 1 – 10), and a given number of hiders (vertical axis, 1 – 5), regardless of hiding strategy. specified for the: Skewed probability distribution.